Agent-Ready LLMs

1. Choosing a model for an agent

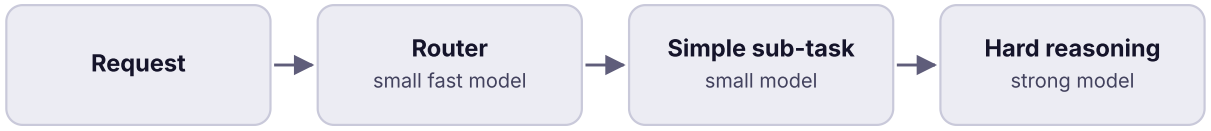
An agent's reasoning core has different needs from a chatbot's. The best benchmark score isn't the criterion; **reliable tool use** usually is.

- **Context window**: must hold system prompt, tool schemas, memory, retrieved context and the running trace
- **Size & specialisation**: a smaller, faster model that routes tools reliably can beat a larger one
- **Native tool calling**: decisive. A model that can't reliably emit valid, correct calls isn't agent-ready
- **Robustness**: resists injection, formats consistently, because it runs unattended in a loop
- **Adaptability**: can it be tuned (PEFT) if the role needs it?
- **Cost & latency**: multiplied by every call in every loop

2. Select per role

- Cheap, fast model → routing, extraction, simple sub-tasks
- Strong model → hard reasoning and planning
- This **tiering** is one of the biggest cost and latency levers you have

✎ DIAGRAM



Route each step to the cheapest model that does it reliably.

3. Serving options

- **Cloud APIs**: fastest start, no ops, elastic; less control over data residency
- **Self-hosted**: full control, data stays in your environment; you own GPUs, serving and upgrades
- **Edge**: small on-device models for privacy, offline use and ultra-low latency

4. Three optimisation axes

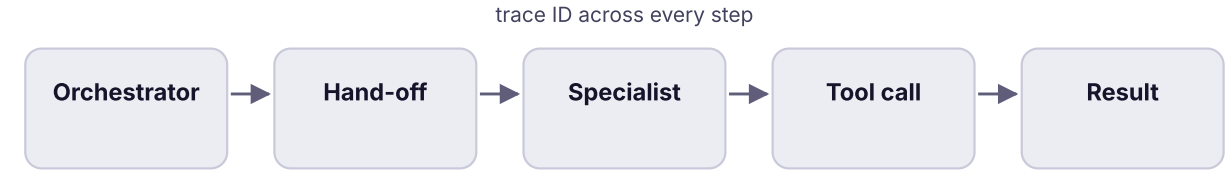
- **Latency**: streaming, smaller or distilled models for simple steps, caching stable prefixes, fewer tool round-trips
- **Throughput**: batching and concurrency
- **Cost**: route easy work to cheaper models, trim context, cap output and reflection loops

> ⚠ In agents, latency is often dominated by **tool round-trips**, not the model. Make tools fast, idempotent and coarse enough to avoid chatty back-and-forth.

5. AgentOps

- LLMOps extended for autonomy: log every step of the loop, trace tool calls and inter-agent messages, version prompts and tools, manage deploy / update / rollback, and watch cost and reliability
- Without **step-level observability**, a multi-agent failure is undebuggable: you need a trace across orchestrator and specialists to see which agent, tool or hand-off caused it

✎ DIAGRAM



Step-level tracing is what makes a multi-agent failure debuggable.

SAY IT

"I select a model per role: a cheap one for routing and simple steps, a strong one for hard reasoning, and I trace every step so failures are debuggable."